Instituto Tecnológico
y de Estudios Superiores de Occidente.
Reconocimiento de validez oficial de estudios de nivel superior según acuerdo secretarial 15018, publicado en el Diario Oficial de la Federación del 29 de noviembre de 1976.

Departamento de Electrónica, Sistemas e Informática
Maestría en Sistemas Computacionales



Deep Learning Sensor Fusion of a MARG IMU and GPS Velocity for 3-D Pose Estimation

---

TRABAJO RECEPCIONAL que para obtener el **GRADO** de
MAESTRO EN SISTEMAS COMPUTACIONALES

Presenta: **ING. ABRAHAM RODRÍGUEZ VÁZQIEZ**
Asesor: **MTRO. MARTÍN ALONSO SINSEL DUARTE**

Tlaquepaque, Jalisco. Octubre de 2026

ACKNOWLEDGEMENTS

I thank Eng. Alberto Arjona, for having been a great source of inspiration and guidance at the beginning of my career in the software industry.

I thank M.Sc. Martín Sinsel, for his patience and constancy, and for having believed in me, even though I was coming from outside the field of electronic engineering.

I thank ITESO, for having left in me a spirit of service, since with time, I have come to see that there is no better way to discover, to grow and to have an influence, than by serving.

AGRADECIMIENTOS

Agradezco al Ing. Alberto Arjona, por haber sido de gran inspiración y guianza en los inicios de mi carrera en la industria del software.

Agradezco al Mtro. Martín Sinsel, por su paciencia y constancia, y por haberme tenido fe, aunque fuese ajeno al área de ingeniería electrónica.

Agradezco al ITESO, por haber embebido en mí, el espíritu de servicio, ya que, tiempo después, me consta que no hay mejor forma de descubrir, crecer e influir, que sirviendo.

DEDICATION

I dedicate this thesis to my daughter Nathalie, for having taught me that life is a gift.

DEDICATORIA

Dedico la presente tesis a mi hija Nathalie, por haberme enseñado que la vida es un regalo.

ABSTRACT

This thesis presents a deep learning approach to the sensor fusion of a magnetic, angular-rate and gravity (MARG) inertial unit and a GPS velocity, with the aim of estimating a 3-D position and a unit quaternion without retrieving the process and measurement noise of a Kalman filter, and without feeding the network a ground-truth previous pose. A one-second window of accelerometer, gyroscope, magnetometer and GPS-velocity measurements is mapped by a dual-head temporal convolutional network to a residual velocity and a short-horizon orientation increment; those increments are then integrated and compared, on the same simulated *LoggedQuadcopter* stream, with MathWorks' *insfilterMARG* extended Kalman filter, using absolute and relative trajectory error and a mean quaternion distance. On a chronological TEST holdout the learned estimator reaches 1.12 m ATE, 0.31 m RTE and 4.95°, i.e. it beats the filter on orientation (7.46°) and ties open-loop GPS-velocity integration on ATE (1.10 m), while the filter remains better on position, as expected, because it is still given GPS latitude, longitude and altitude, which the network never sees. A recursive closed-loop network and a hybrid residual on the frozen EKF were implemented as design alternatives and are not the defended method. The work is simulated and limited to a single trajectory; Chapter 2 surveys the related techniques and the three-way fork, Chapter 4 describes the increment pipeline, Chapter 5 reports the comparison, and Chapter 6 states the conclusions and the recursive line as future work.

RESUMEN

Esta tesis presenta un enfoque de aprendizaje profundo para la fusión de un sensor inercial MARG (magnético, de velocidad angular y de gravedad) y de una velocidad GPS, con el fin de estimar una posición 3D y un cuaternión unitario, sin recuperar las covarianzas de ruido de un filtro de Kalman y sin alimentar a la red con la pose verdadera anterior. Una ventana de un segundo de acelerómetro, giroscopio, magnetómetro y velocidad GPS es mapeada por una red convolucional temporal de dos cabezas a una velocidad residual y a un incremento de orientación de corto horizonte; esos incrementos se integran y se comparan, sobre la misma trayectoria simulada *LoggedQuadcopter*, con el filtro de Kalman extendido *insfilterMARG* de MathWorks, mediante el error absoluto y relativo de trayectoria y una distancia media de cuaterniones. En una partición cronológica de prueba, el estimador aprendido alcanza 1.12 m de ATE, 0.31 m de RTE y 4.95°, es decir, supera al filtro en orientación (7.46°) y empata la integración en lazo abierto de la velocidad GPS en ATE (1.10 m), mientras que el filtro sigue siendo mejor en posición, como es de esperarse, porque aún recibe latitud, longitud y altitud, que la red nunca ve. Una red recursiva en lazo cerrado y un residual híbrido sobre el EKF congelado se implementaron como alternativas de diseño y no constituyen el método que se defiende. El trabajo es simulado y se limita a una sola trayectoria; el Capítulo 2 revisa las técnicas relacionadas y la bifurcación de tres opciones, el Capítulo 4 describe el pipeline de incrementos, el Capítulo 5 reporta la comparación y el Capítulo 6 plantea las conclusiones y la línea recursiva como trabajo futuro.

CONTENT

MAESTRÍA EN SISTEMAS COMPUTACIONALES 1

1. INTRODUCTION 13

1.1. BACKGROUND 13
1.2. JUSTIFICATION 13
1.3. PROBLEM 14
1.4. HYPOTHESIS 14
1.5. OBJECTIVES 15
1.5.1. General Objective 15
1.5.2. Specific Objectives 15
1.6. SCIENTIFIC OR TECHNOLOGICAL NOVELTY 15

2. STATE OF THE ART 16

2.1. CLASSICAL GNSS/INS INTEGRATED NAVIGATION 16
2.2. LEARNING-BASED INERTIAL ODOMETRY 17
2.3. HYBRID LEARNING AND FILTERING 18
2.4. DEEP LEARNING FOR GNSS/INS OUTAGE AIDING 18
2.5. SYNTHESIS AND THE THREE DESIGN OPTIONS 18

3. THEORETICAL FRAMEWORK 21

3.1. REFERENCE FRAMES AND ATTITUDE 21
3.2. INERTIAL AND SATELLITE SENSORS 22
3.3. CLASSICAL SENSOR FUSION 23
3.4. DEEP LEARNING MODELS USED IN THIS WORK 24
3.5. TRAINING, LOSSES AND OPTIMIZATION 25
3.6. DEAD RECKONING, TEACHER FORCING AND SCHEDULED SAMPLING 26
3.7. EVALUATION METRICS 27

4. METHODOLOGICAL DEVELOPMENT 28

4.1. DESIGN ALTERNATIVES FOR THE LEARNED ESTIMATOR 28

4.2. Simulated dataset and the classical baseline 29
4.3. Defended estimator: increment TCN 30
4.4. Recursive and hybrid pilots 33
5. RESULTS AND DISCUSSION 34
5.1. Results 34
5.2. Discussion 37
6. CONCLUSIONS 40
6.1. Conclusions 40
6.2. Future work 41

FIGURES

Figure 1. Chronological train / validation / test split of the *LoggedQuadcopter* log. One-second windows are extracted every 0.1 s and position is integrated every 1.0 s; one-second gaps separate the splits so that no window is shared. 30

Figure 1. Defended Option A pipeline. A one-second, twelve-channel MARG-plus-GPS-velocity window is mapped by a dual-head temporal convolutional network to a residual velocity and a 0.1 s axis-angle increment; the increments are integrated into a pose and compared with *insfilterMARG* and GPS-velocity integration. 31

Figure 1. TEST scorecard. A8q (red) versus the rejected A8 ATE checkpoint, *insfilterMARG* and GPS-velocity integration. Quaternion is the number A8q wins; ATE is a tie with GPS-vel. 35

Figure 2. Integrated TEST path in NED and in a north-east top-down view. A8q follows the quadcopter; the EKF stays closer in position because it sees GPS LLA. 35

Figure 3. Position and quaternion error versus time. Position error grows from the first 10 s to the last 10 s; quaternion stays below the EKF for most of the holdout. 36

Figure 4. One-second mean velocity. The leftover bias of about 0.09 m/s is what walks ATE by a meter over the twenty-second TEST. 36

TABLES

Table 1. Comparison of related learned-inertial and hybrid methods: input, output, whether a Kalman filter is kept, GPS use, and what occupies the previous-state slot (TF = teacher forcing). 19

Table 1. Reproducibility settings for the defended A8q increment TCN: data, windowing, chronological split, architecture, and training. 32

Table 1. TEST metrics, 121–141 s, versus ground truth. A8q is the defended row. B1 and C0 are pilots. 34

ACRONYMS AND ABBREVIATIONS

ATE	Absolute Trajectory Error
CNN	Convolutional Neural Network
DL	Deep Learning
EKF	Extended Kalman Filter
GNSS	Global Navigation Satellite System
GPS	Global Positioning System
GT	Ground Truth
IMU	Inertial Measurement Unit
INS	Inertial Navigation System
KF	Kalman Filter
LLA	Latitude, Longitude, Altitude
LSTM	Long Short-Term Memory
MARG	Magnetic, Angular Rate, and Gravity
MLP	Multilayer Perceptron
MSE	Mean Squared Error
NED	North-East-Down
NLL	Negative Log-Likelihood
RMSE	Root Mean Square Error
RTE	Relative Trajectory Error
SS	Scheduled Sampling
TCN	Temporal Convolutional Network
ZOH	Zero-Order Hold

1. INTRODUCTION

Summary: This chapter frames the problem of fusing a magnetic, angular-rate and gravity (MARG) inertial unit with a consumer GNSS receiver, states why a learned increment model is worth writing down next to a classical extended Kalman filter and records the objectives and the novelty claim that the rest of the document has to defend.

1.1. Background

As with many sensing problems that used to live entirely inside a filter designer's notebook, inertial/GNSS fusion has gone through a transitional period in which the same job, i.e. turning noisy accelerometers, gyroscopes and satellite fixes into a pose, is now also attempted with sequence models that never write down a process-noise matrix; the classical line is still the extended Kalman filter and its error-state cousins, of which MathWorks' *insfilterMARG* is the concrete baseline used in this work, while the learning line, from *RIDI* and *IONet* through *RoNIN* and *OriNet*, replaces double integration with a windowed regression of displacement or velocity [1]–[4]. A third family keeps the filter and trains a network to supply a residual, a covariance, or a pseudo-GNSS measurement, which is the pattern of *TLIO*, *AI-IMU*, Al-Sharman and Wang [5]–[8]. This thesis sits in that landscape with a full MARG suite plus GPS velocity, a 3-D position and quaternion target, and a simulated stream generated by the same *insfilterMARG* example that later serves as the numerical opponent; the earlier course project that fed a ground-truth previous pose into the net is treated here as a rejected setup, not as the method to defend.

1.2. Justification

The practical pressure is cost and time: a well-tuned EKF can be excellent on a given IMU, and it can also become a week of Q and R archaeology the moment the sensor, the mounting, or the motion statistics change, whereas a learned increment model, if it works, is a promise that those variances do not have to be retrieved by hand. I consider that the more interesting question for a master's thesis is not whether deep learning can be stapled onto a Kalman filter, that path is already the 2020–2025 paper farm [5]–[8], but whether a Q/R-free

network that never sees the true pose can be compared, honestly, with the filter that generated its training noise, on the same trajectory and with the same metrics the inertial-odometry literature already uses, i.e. absolute and relative trajectory error plus a quaternion distance [3], [4]. There is also a narrower, local justification: the hardware side of this work, an ESP32 logging a BNO055 and a NEO-M8N, still needs a fusion story that does not collapse the first time the process-noise table is copied from a datasheet.

1.3. Problem

The practical problem is to estimate a 3-D position and a unit quaternion from a 160 Hz MARG stream and a 1 Hz GPS velocity, without assuming that the previous true pose will be available at runtime. The scientific problem is to do that as learned dead reckoning, i.e. to regress a local increment and integrate it, and to report the integrated path against *insfilterMARG* with ATE, RTE and mean quaternion error, rather than with a teacher-forced one-step MSE that hides drift. A secondary problem, which this document does not pretend to have solved, is that a recursive net that feeds back its own pose can look almost perfect while it is still being handed the ground truth, and can come apart the moment that crutch is removed.

1.4. Hypothesis

A dual-head temporal convolutional network that maps a one-second window of MARG measurements and GPS velocity to a residual velocity and a 0.1 s axis-angle increment, with no ground-truth pose in the inputs, can be integrated to produce a TEST trajectory whose mean quaternion error is lower than that of *insfilterMARG* on the same stream, and whose ATE is at least comparable to open-loop integration of the GPS velocity itself.

1.5. Objectives

1.5.1. General Objective

To design, train and evaluate a learned increment estimator that fuses MARG inertial measurements with GPS velocity, integrates the predicted increments into a pose, and compares that pose with a classical MARG/GNSS extended Kalman filter on the same simulated trajectory, without feeding the ground-truth previous state and without retrieving the filter's Q and R by hand.

1.5.2. Specific Objectives

The specific objectives are: 1) to generate a reproducible MARG-plus-GPS-velocity dataset from the *LoggedQuadcopter* trajectory and *insfilterMARG*, with the filter's own pose logged as a baseline; 2) to train a dual-head temporal convolutional network on residual velocity and short-horizon orientation increments, using a chronological holdout; 3) to integrate those increments and report ATE, RTE and mean quaternion distance against the EKF and against GPS-velocity dead reckoning; 4) to document the recursive closed-loop alternative and the hybrid residual-on-EKF alternative, and to keep them out of the defended comparison table.

1.6. Scientific or technological novelty

I do not claim a new class of neural filter. The increment idea is already in *IONet*, *RIDI* and *RoNIN* for IMU-only pedestrians [1]–[3]; what this work adds, and what I consider the actual contribution at master's scope, is a MARG-plus-GPS-velocity increment model with a 3-D position and quaternion head, trained and scored against the same *insfilterMARG* instance that produced the sensor noise, with teacher-forced previous pose explicitly rejected. The recursive net that consumes its own last velocity and quaternion is the most vacant niche in the reading list, and it is reported as a closed-loop pilot, not as the defended method; the hybrid that adds a learned leftover to the EKF is documented because it is the pattern the papers already built [5]–[8], not because it is offered as novelty.

2. STATE OF THE ART

Summary: This chapter walks from classical GNSS/INS fusion to learned inertial odometry and to the hybrids that keep a Kalman filter in the loop, and it closes on the three design options that were actually available for this thesis; the point of the survey is to show where the defended increment model sits, and why the most published hybrid pattern was not treated as the automatic "modern" choice.

2.1. Classical GNSS/INS integrated navigation

The textbook solution for combining an inertial measurement unit with satellite observations is still a Kalman filter, in the linear case, or an extended Kalman filter once the attitude and the specific-force rotation make the dynamics nonlinear; Kalman stated the discrete predict/update recursion in 1960 [10], and Groves is the reference this work uses for frames, inertial error models, and the loosely versus tightly coupled GNSS/INS taxonomy [11]. In the loosely coupled picture the GNSS receiver produces a position and a velocity in a navigation frame, those observations update an INS that has been integrating accelerometer and gyroscope measurements, and the designer's real labor is the pair of covariance matrices Q and R, i.e. how much the prediction is trusted versus how much the satellite fix is trusted.

A magnetic, angular-rate and gravity (MARG) unit adds a magnetometer to the usual six inertial axes, which gives the filter an absolute heading observation that a gyro-only integrator does not have; MathWorks' *insfilterMARG* is an error-state EKF built for that sensor suite plus GPS, and it is the classical baseline of this thesis because it is also the generator of the simulated measurements. Over a 50 s excerpt of the official example the filter reaches position RMSE values on the order of half a meter per axis and a quaternion distance of about 1.45°; on the full *LoggedQuadcopter* log used here, roughly 142 s after warmup, the same filter, scored on the TEST slice with the same ATE/RTE protocol as the network, sits at 0.78 m ATE, 0.20 m RTE and 7.46° mean quaternion error. Those numbers are the opponent, not a footnote, and they already include GPS position (latitude, longitude, altitude) in the update, which the learned increment model is never given.

It is important to note that a well-tuned EKF is not a weak baseline, and that "learning replaces the filter" is only an interesting sentence if the comparison is run on the same stream, with the same holdout, and with metrics that punish drift rather than one-step pose MSE. The rest of this chapter is the set of published alternatives to writing Q and R by hand.

2.2. Learning-based inertial odometry

The learning-based line starts from a simple observation, which Chen et al. put in the abstract of *IONet*: continuous integration is a curse, because a small bias becomes a walk, and the way out is to cut the inertial stream into independent windows and regress a local displacement [1]. *RIDI* had already acted on the same instinct with shallow models, an SVM on phone placement and an SVR on gravity-aligned 2-D velocity, then a least-squares bias correction and double integration, and it left deep networks as explicit future work [2]. *RoNIN* turned that recipe into a large in-the-wild benchmark and three backbones, ResNet, LSTM and a temporal convolutional network, still IMU-only, still 2-D velocity, still without a filter at inference, and it is the paper from which this thesis takes ATE and RTE as the path metrics [3].

OriNet is the closest published cousin on the orientation side: a multi-LSTM plus MLP that regresses a quaternion from gyroscope samples, a time step, and the previous quaternion, and the authors themselves describe the net as acting like an EKF that updates from the last state [4]. The previous-quaternion input gave them a large gain over their own earlier model; it is also, if that previous quaternion is the ground truth, teacher forcing, i.e. the network is never asked to live with a heading it just got wrong. I consider that distinction more important than the architecture diagram, because a master's table that reports a one-step quaternion MSE under teacher forcing is answering a different question from "can this thing fly."

Almost all of the increment literature above is IMU-only, magnetometer disabled, no GPS, and trained on real pedestrian or MAV logs. The gap that is actually available, once teacher-forced previous pose is taken off the table, is narrower and cleaner: a full MARG suite plus GPS velocity, a 3-D increment and a quaternion, scored against *insfilterMARG* on the same simulation. That is not a new paradigm; it is a fair increment on [1]–[3] with a heading head that [4] already showed was learnable.

2.3. Hybrid learning and filtering

Hybrid approaches combine hand-crafted algorithms and learning models [9]; in inertial navigation that sentence has a very concrete meaning, the Kalman filter stays, and the network is hired to do the part a human used to tune. *TLIO* regresses a 3-D displacement and an uncertainty from a 1-D ResNet and feeds both into a stochastic-cloning EKF [5]. *AI-IMU* keeps an invariant EKF and trains a small 1-D CNN only to adapt the pseudo-measurement noise, i.e. the network is the R schedule, not the state [6]. Al-Sharman trains a DNN on the Kalman filter's own attitude so that residual noise can be subtracted from a filter that already ran [7]. Wang et al. push the same idea into GNSS outage: a Transformer–LSTM predicts pseudo-GNSS increments that a 15-state KF then consumes [8].

This is the developed 2020–2025 line, and it is also the easiest thesis to write if the only goal is to look contemporary. I thought, at the beginning of this work, that "put the LSTM on the EKF" was the sophisticated option; after reading [5]–[8] it became clear that it is the occupied option. A hybrid can still be a useful pilot, and one was implemented, but it cannot be sold as the novelty of this document.

2.4. Deep learning for GNSS/INS outage aiding

Wang et al. are the closest GNSS/INS-plus-deep-learning analog in the reading list: the network sees a 12-D inertial and INS-derived window and predicts a 4-D GNSS pseudo-increment, which is a learned measurement, not a learned pose, and the filter remains the thing that closes the loop during a 50 s outage [8]. The headline reductions versus pure INS are large, as they should be when a satellite-like measurement is reintroduced; the methodological lesson for this thesis is different, namely that predicting a GNSS-like observation to feed a KF is already a published pattern, and repeating it with *insfilterMARG* would be an implementation of that pattern.

2.5. Synthesis and the three design options

Chen et al.'s survey organizes localization into model-based, learning-based and hybrid families [9], and that trichotomy is exactly the fork this project had to pick from once the original teacher-forced previous-state network was discarded. Teacher forcing, in this document, means that during training the previous pose fed to the model is the ground-truth

pose at t−1; the net never sees its own error, the loss is easy, and the number that comes out overstates in-flight drift. *OriNet* uses a previous quaternion in that spirit [4]. That setup is kept only as a diagnostic print, never as the claim. **Table 1** contrasts these methods with the present work along input, output and whether a filter is kept.

Table 1. Comparison of related learned-inertial and hybrid methods: input, output, whether a Kalman filter is kept, GPS use, and what occupies the previous-state slot (TF = teacher forcing).

Method	Input	Output	Filter kept	GPS	Previous state
IONet [1]	IMU (acc, gyro)	2-D displacement	No	No	none (windowed)
RIDI [2]	IMU	2-D velocity	No	No	none
RoNIN [3]	IMU	2-D velocity	No	No	none
OriNet [4]	Gyroscope	Quaternion	No (EKF-like)	No	previous quaternion (TF)
TLIO [5]	IMU	3-D displ. + unc.	Yes (EKF)	No	filter state
AI-IMU [6]	IMU	Meas. noise R	Yes (IEKF)	No	filter state
Al-Sharman [7]	KF attitude	Attitude residual	Yes (KF)	No	filter state
Wang et al. [8]	IMU + INS window	GNSS pseudo-incr.	Yes (KF)	GNSS (outage)	filter state
This work (A)	**MARG + GPS velocity**	**δv + 0.1 s $\Delta\theta$**	**No**	**GPS velocity**	**none**

Option A drops the previous pose entirely. The network sees MARG and GPS velocity, regresses a local increment, residual velocity plus a short-horizon axis-angle, and those increments are integrated; there is no Q and no R, and the comparison with *insfilterMARG* is a comparison of two estimators on the same measurements, except that the EKF is still allowed to use GPS position, which A is not. The idea is *IONet*/*RoNIN*-style learned dead reckoning [1], [3], extended to magnetometer, GPS velocity, and a quaternion.

Option B keeps a previous state, but it is the network's own last prediction, and training is closed-loop, with scheduled sampling as the on-ramp from ground truth to self-generated pose [17]. The net is the filter; *OriNet* is the cousin, except *OriNet* teacher-forces a gyro-only quaternion and has no GPS [4]. Among the three options this is the least occupied niche in the paper set, and it is also the one whose wheels come off, which is why it is written as future work with a pilot, not as the main table.

Option C keeps *insfilterMARG* and trains a network on the leftover, a residual and a noise scale, or, in the papers, a covariance or a pseudo-GNSS increment [5]–[8]. Q and R come back unless the net replaces them. This is the safest implementation and the weakest

novelty claim; the document records a C0 pilot so that the fork is empirical, not rhetorical, and it does not present C as the defended method.

The novelty ranking inside this reading list is therefore the inverse of the publication count: B is the most open and the hardest to train, A is a thinner but honest increment on IMU-only odometry, and C is the pattern that already has a venue trail. The choice, which Chapter 4 has to justify in engineering terms, is A for the main thesis, B for the recursive extension, and C as related work that was run once so the residual-on-EKF story would have numbers.

3. THEORETICAL FRAMEWORK

Summary: This chapter is a term-by-term account of the quantities, sensors, filters, networks and metrics that the rest of the document uses, so that Chapter 4 can name a residual velocity or a quaternion distance without stopping to define them; the tone is that of a glossary, not of a second state of the art, and each term is tied to the citation that this work actually leans on.

3.1. Reference frames and attitude

Navigation is not done in a single set of axes, and mixing them is how a filter, or a network, writes a number that looks precise and points the wrong way; Groves is the reference used here for the geographic, navigation and body frames [11].

Geographic frame. A position on the Earth is written as latitude, longitude and altitude, often abbreviated LLA; GPS receivers report this frame, and *insfilterMARG* consumes it in the update. The defended network never sees LLA, only a velocity already expressed in the navigation frame, because a held latitude in the feature vector would turn absolute trajectory error into a contest the filter had already been given the answer to.

NED. The North–East–Down navigation frame is the working frame of this thesis: the first axis points north, the second east, and the third down, so that a positive increment in the third coordinate is a descent. Ground-truth position, GPS velocity and the EKF position in *filterstate.csv* are all stored in NED, following the MathWorks MARG example [18].

Body frame. The accelerometer, gyroscope and magnetometer measure in the vehicle body, i.e. in axes fixed to the IMU. A specific force has to be rotated by the current attitude before it can be integrated as a change of NED velocity; that rotation is the reason attitude and position cannot be learned as two unrelated regressions if one later wants a consistent path [11].

Euler angles. Roll, pitch and yaw are an intuitive three-number attitude, and they are also a poor regression target near pitch ±90°, where the representation is singular. This work does not train on Euler angles.

Quaternion. A unit quaternion q = (q_w, q_x, q_y, q_z) is a four-vector on the sphere that represents a rotation without that singularity; the product of two quaternions is another rotation, and the inverse of a unit quaternion is its conjugate [11], [4]. The ground-truth and EKF orientations in the dataset are stored in wxyz order. Composition of a previous attitude q with a small increment Δq is written

$$q_k = q_{k-1} \otimes \Delta q_k \tag{1}$$

and the increment itself is obtained, in this work, from a three-dimensional axis-angle vector v = θ u, with θ the rotation angle and u a unit axis, by the map

$$\Delta q = (\cos(\theta/2), u\sin(\theta/2)) \tag{2}$$

Quaternion distance. Because q and −q are the same rotation, a useful scalar error is the geodesic angle used by *OriNet* and kept in Chapter 5 [4],

$$\theta_D = 2\arccos(|\hat{q} \cdot q|) \tag{3}$$

reported in degrees as a mean over the TEST sequence. A 180° value that never moves, as in the first increment experiment of this project, is a sign that the network has collapsed to an antipodal quaternion, not that the vehicle flipped.

3.2. Inertial and satellite sensors

IMU. An inertial measurement unit, in the six-axis sense, is an accelerometer plus a gyroscope; the accelerometer reports specific force, i.e. acceleration minus gravity in the body frame, and the gyroscope reports angular rate. Integrating those signals is dead reckoning, and it walks off in seconds on a consumer IMU because of bias, noise density and scale-factor error [11].

MARG. A magnetic, angular-rate and gravity unit adds a magnetometer to the IMU, which gives an absolute heading observation that a gyro-only integrator does not have. The simulated sensor in this work is MATLAB's *imuSensor* in the accel-gyro-mag configuration, with the noise densities and constant biases of the official example [18]; the twelve-channel learning window is accelerometer (3), gyroscope (3), magnetometer (3) and GPS velocity (3).

Sensor noise, bias and density. The example IMU is not a white-noise toy: the accelerometer and gyroscope carry a constant bias and a noise density, the magnetometer carries a large constant bias, and those parameters are also used to seed the EKF state. I consider that this is why a Q/R-free network is interesting on this log, and also why it is not a claim about an unseen physical IMU: the noise the net sees is the noise the example already wrote down.

GNSS and GPS velocity. A global navigation satellite system receiver, here a simulated *gpsSensor* standing in for a consumer GPS, produces a geodetic position and a velocity at 1 Hz, with a horizontal and vertical position accuracy of 1.6 m and a velocity accuracy of 0.1 m/s in the generator script. Velocity is zero-order held into the 160 Hz table so that the network always sees a 12-D sample; position LLA is withheld from that table and reserved for the EKF update. Open-loop integration of the held GPS velocity is the naïve baseline of Chapter 5, i.e. what the residual head must beat or tie before anyone talks about beating the filter.

Zero-order hold (ZOH). A 1 Hz measurement that is repeated until the next fix, so that a 160 Hz learner has a value on every row. ZOH of magnetometer and GPS velocity is used; ZOH of LLA is not, for the reason already stated.

3.3. Classical sensor fusion

The classical estimator in this document is an extended Kalman filter. Kalman stated the linear recursion in 1960 [10]; Groves is the navigation reading [11]; *insfilterMARG* is the concrete MARG-plus-GPS implementation that both generates the comparison pose and, in the hybrid pilot, supplies the leftover that a network may try to subtract [18].

State-space model. A discrete process and a measurement are written

$$x_k = f(x_{k-1}, u_k) + w_k, z_k = h(x_k) + v_k \tag{4}$$

where w_k is process noise with covariance Q and v_k is measurement noise with covariance R. The designer's labor, when there is no network, is those two matrices.

Kalman filter. In the linear case the predict step propagates the mean and the covariance, and the update step computes a gain K that blends the prediction with the measurement [10],

$$\hat{x}_{k|k-1} = F\hat{x}_{k-1|k-1}, P_{k|k-1} = FP_{k-1|k-1}F^T + Q \tag{5}$$

$$K_k = P_{k|k-1}H^T(HP_{k|k-1}H^T + R)^{-1} \tag{6}$$

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k(z_k - H\hat{x}_{k|k-1}) \tag{7}$$

Q and R. Q says how much the prediction is trusted; R says how much the sensor is trusted. Swapping their names in a write-up, or putting a magnetometer variance into R as if it were already an angle, is a known way to get a filter that runs and a text that does not match the code. This thesis does not retune Q or R; the example values travel with the simulated IMU.

EKF and *insfilterMARG*. An extended Kalman filter linearizes f and h about the current estimate. *insfilterMARG* is an error-state EKF for a 22-dimensional MARG-plus-GPS state, including attitude, NED position and velocity, inertial biases and the geomagnetic vector [18]. On the TEST slice used in Chapter 5 it reaches 0.78 m ATE, 0.20 m RTE and 7.46° mean quaternion error. Those numbers already include GPS LLA in the update.

Loosely versus tightly coupled fusion. A loosely coupled GNSS/INS filter consumes the receiver's position and velocity solutions; a tightly coupled filter consumes raw or processed satellite observables [11]. The baseline of this work is loosely coupled in that sense: *fusegps* is called with LLA and velocity. The learned estimator is looser still: it sees only the velocity channel.

3.4. Deep learning models used in this work

The defended estimator is a temporal convolutional network with two linear heads. The terms below are only those that appear in that pipeline or in the alternatives of Chapter 2; Goodfellow et al. are the textbook backdrop [12].

MLP. A multilayer perceptron is a stack of affine maps and pointwise nonlinearities; here each head is a two-layer MLP, 128–64–3, on top of a pooled TCN body [12].

ReLU. The rectified linear unit, max(0, x), is the nonlinearity inside the TCN blocks and the heads.

Conv1d. A one-dimensional convolution slides a kernel along time, across channels; it is the natural layer for a window of IMU samples, and it is the stem of both the increment model and the older Conv1d–LSTM course network [15], [12]. A kernel of length 5 and 64 channels is the stem used in A8q.

TCN. A temporal convolutional network is a stack of dilated residual 1-D convolutions whose receptive field grows exponentially with depth. *RoNIN* already used a TCN as one of three inertial backbones [3]. The A8q body has four residual blocks with dilations 1, 2, 4 and 8, and a kernel of length 5 with symmetric padding, so this stack is not a strictly causal TCN; the representation fed to the heads is the concatenation of a global average over time and the last time step.

Residual velocity head. The position-related output is not a NED position and not a raw velocity, it is a correction δv such that

$$\hat{v} = v_{\mathrm{GPS}} + \delta v \tag{8}$$

so that a network that outputs zero still reproduces GPS-velocity dead reckoning. The 0.1 s orientation head outputs the axis-angle of (2), not a unit quaternion, because a 160 Hz Δq is almost the identity and was observed to collapse to 180° in the first increment run.

LSTM. A long short-term memory cell is a recurrent unit with input, forget and output gates [13]. It is the backbone of *IONet* and *OriNet* [1], [4], and of the course-project network that this thesis retired; it is not the frozen A8q body. It is defined here because Chapter 2 cannot be read without it.

Dual head. Two small MLPs on a shared trunk, one for δv and one for the axis-angle. A single six-dimensional head was tried; ATE pressure spent the orientation. I consider that split the one architectural decision that actually changed the table, more than the width of the TCN.

3.5. Training, losses and optimization

Train / validation / test split. Windows are extracted every 0.1 s from a one-second context. The log is cut chronologically, about 70 / 15 / 15 with a one-second gap, so that TRAIN, VAL and TEST do not share a hop; TEST is 121–141 s. This is a same-path holdout, not a claim that the net generalizes to another flight [12].

Extract versus hop. Windows are cut every 0.1 s, that is the extract, but position is integrated every 1.0 s, that is the hop, so that a one-second mean velocity is not applied ten times per second; orientation still uses the 0.1 s extract. Mixing those two clocks was how the early increment runs either starved the train set or walked a good local velocity into a bad path.

Normalization. Feature-wise and target-wise means and standard deviations are computed on TRAIN only and applied to VAL and TEST; without that scaling the position term dominates a joint loss and the orientation is under-learned.

MSE. Mean squared error on the standardized residual velocity and on the axis-angle is the training loss of A8q [12]. A geodesic loss on (3) was left as a possible ablation and was not used for the frozen row.

Negative log-likelihood (NLL). The C0 pilot is not trained with that MSE; it predicts a leftover mean μ and a log-standard-deviation and minimizes a Gaussian $-\log p(\text{error} \mid \mu, \sigma)$ on the frozen EKF residual. The term is here only so Chapter 5 can name the hybrid number without inventing a fifth loss.

Adam. The optimizer is Adam at 10^{-3} [14], with minibatches of 256 and eighty epochs. Early stopping, in practice, is a validation checkpoint, and the checkpoint metric decides which compass one keeps.

Dropout. Randomly dropping units during training as a regularizer [16]. It is cited because Al-Sharman uses it and because the course network sketched it; A8q does not rely on it as the main control of overfitting, the chronological split and the dual head do more of that work.

3.6. Dead reckoning, teacher forcing and scheduled sampling

Dead reckoning. Estimating the present pose from the last pose and a measured or predicted increment, with no external fix. Inertial integration is dead reckoning; *IONet*'s sentence about breaking continuous integration is the learning version of the same idea [1]. The defended estimator is learned dead reckoning plus a GPS-velocity measurement, not a pose prior.

Teacher forcing. During training, the previous state fed to a sequential model is the ground truth, not the model's own last output. The loss becomes easy and the number

overstates closed-loop drift; *OriNet*'s previous quaternion is in that family if that quaternion is the true one [4]. Teacher forcing is rejected as the claim of this thesis and kept only as a diagnostic column in the recursive pilot.

Exposure bias. The mismatch between a teacher-forced train-time input and a self-generated test-time input. B0 made that mismatch theatrical: a 0.07 m teacher-forced ATE and a 2.57 m closed-loop ATE, because the net had been looking up a NED position.

Scheduled sampling. Bengio et al.'s on-ramp: the probability of feeding ground truth is annealed toward zero so that the net is gradually asked to live with its own pose [17]. B1 uses a linear anneal over the first 40% of epochs and then stays at zero.

Closed-loop evaluation. The first TEST pose is taken from ground truth; after that the estimator consumes only sensors and its own last state. A is closed-loop in the integrator, not in the network, because A has no previous-state input. B is closed-loop in both.

3.7. Evaluation metrics

ATE. Absolute trajectory error, taken from *RoNIN*'s evaluation culture [3]: the RMSE of the integrated NED position against ground truth on the 1 s TEST hops,

$$\mathrm{ATE} = \sqrt{\frac{1}{N}\sum_i \| \hat{p}_i - p_i \|^2} \tag{9}$$

A constant 0.09 m/s velocity bias becomes about a meter over a 20 s holdout, so ATE punishes drift. The EKF's ATE is not a fair fight with A, because the filter sees LLA.

RTE. Relative trajectory error over a short window, here one second: the mean magnitude of the difference between estimated and true displacements on that hop [3]. A path that is translated but has the right shape can have a bad ATE and a decent RTE. A8q's claim against GPS-velocity integration is an RTE claim, 0.31 m versus 0.45 m.

RMSE. Root-mean-square error, the scalar cousin of ATE when the quantity is not a trajectory, e.g. a per-axis velocity or a one-step increment.

Those nine numbered relations, and the terms around them, are the vocabulary of Chapters 4 and 5; what is not defined here, a Transformer, an invariant EKF, a stochastic-cloning filter, is named in Chapter 2 and left there.

4. METHODOLOGICAL DEVELOPMENT

Summary: This chapter records the design fork, the simulated dataset, and the defended increment pipeline; the recursive and hybrid estimators are specified only as far as needed to make the later numbers readable, they are not second and third methods competing for the same claim.

4.1. Design alternatives for the learned estimator

Option A is the defended estimator. A one-second window of accelerometer, gyroscope, magnetometer and GPS velocity, twelve channels, no pose, is mapped to a residual 3-D velocity and a 0.1 s axis-angle; position is integrated every second as pos ← pos + $\hat{v}\cdot 1.0$, and the quaternion is integrated every 0.1 s as $q \leftarrow q \otimes \mathrm{quat}(\Delta\theta)$. There is no process-noise table and no measurement-noise table. I consider this the only option that keeps the original taste of the project, i.e. that everything the estimator needs is learned, and that still has a clean published ancestor in windowed increment odometry [1]–[3].

Option B is a learned recursive filter. The same window is concatenated with the last velocity and the last quaternion produced by the net, scheduled sampling anneals the probability of feeding ground truth from one to zero, and TEST is seeded once and then never sees ground truth again [17]. Feeding the absolute NED position was tried and discarded: with the true position in its hand the teacher-forced score collapsed to centimeters, which is a lookup of the training trajectory, not a filter. B is the option that still honors “no Q/R” and is not already the plot of [5]–[8]; it is also the option that can look healthy on a one-second unroll and sick on a twenty-second flight, which is why it is not the main-table method.

Option C is the hybrid. *insfilterMARG* is left frozen, including the example’s *AccelerometerNoise*, *Rpos* and *Rvel*, and a network predicts the leftover error μ and a log-standard-deviation, trained with a Gaussian negative log-likelihood; the corrected pose is the EKF pose plus μ. That is Al-Sharman on this filter, and it is a stand-in for “the model is the noise” without rewriting Q in MATLAB and rerunning the example [6], [7]. A full *AI-IMU* loop is not claimed.

The decision was A. The argument is not that A is the most novel, B is, and it is not that A is the easiest to make look good on ATE, C is, because C inherits GPS latitude, longitude and altitude through the filter. A is the option that can be compared with the EKF without pretending to be the EKF, that does not require a previous true pose at runtime, and that still lets B and C be written down as the recursive extension and the already-published hybrid. Teacher forcing remains a diagnostic column in B, so the exposure-bias gap is visible, and it is not sold as a result.

4.2. Simulated dataset and the classical baseline

The dataset is generated in MATLAB from *LoggedQuadcopter*.mat, using *imuSensor* and *gpsSensor* at 160 Hz and 1 Hz, and *insfilterMARG* as both the noise-consistent classical estimator and the source of the filter-state log. Magnetometer and GPS velocity are zero-order held at 1 Hz in the learning table; GPS latitude, longitude and altitude are consumed by the EKF and are not written into the twelve-channel stream, because putting a held LLA into the network would turn ATE into a competition the EKF had already been given the answer to. The first second of warmup is dropped. Ground-truth position is in NED; ground-truth orientation is a unit quaternion in wxyz order. The MATLAB script is recorded in Appendix A.

Windows are one second long, extracted every 0.1 s. After the chronological 70 / 15 / 15 cut and a one-second gap the counts are 961 TRAIN, 210 VAL and 210 dense TEST windows, with TRAIN occupying roughly 1–97 s, VAL 99–120 s and TEST 121–141 s; calling this “generalization” would be dishonest, it is a same-path holdout. Feature and target statistics are computed on TRAIN only. The EKF pose used by C and by the baseline columns is read from *filterstate.csv*, aligned to the same window-end samples. The resulting split is summarized in **Figure 1**.

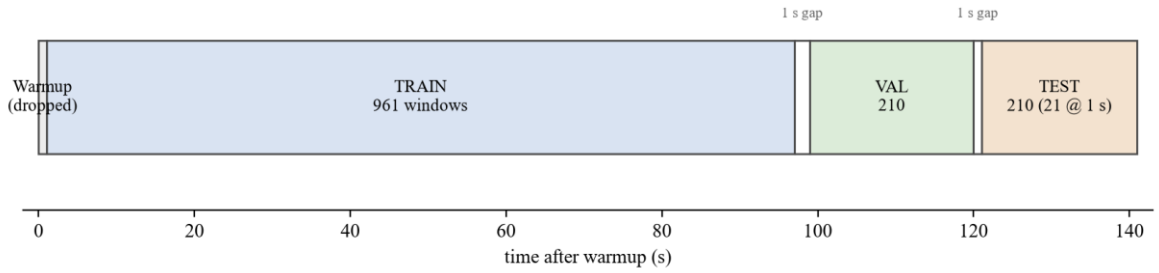


Figure 1. Chronological train / validation / test split of the *LoggedQuadcopter* log. One-second windows are extracted every 0.1 s and position is integrated every 1.0 s; one-second gaps separate the splits so that no window is shared.

4.3. Defended estimator: increment TCN

The backbone is a temporal convolutional network rather than the course-project Conv1d–LSTM: a 12-to-64 stem, four residual blocks with kernel 5 and dilations 1, 2, 4 and 8, then a concatenation of global average pooling and the last time step, 128 units, and two heads, each 128–64–3, one for residual velocity and one for the 0.1 s axis-angle. A shared six-dimensional head was tried; position pressure spent the orientation, which is why the heads were split. The residual form is $\hat{v} = v_{gps} + \delta v$, so that a network that outputs zero still reproduces GPS-velocity dead reckoning, and any ATE gain has to be earned against that baseline, not against a cold start. The full pipeline is shown in **Figure 2**.

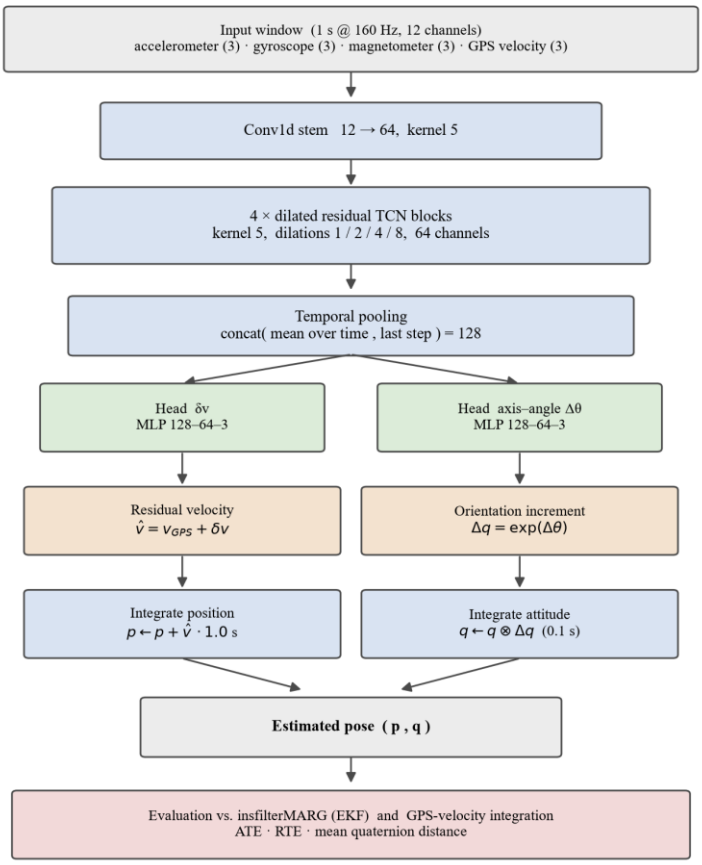


Figure 2. Defended Option A pipeline. A one-second, twelve-channel MARG-plus-GPS-velocity window is mapped by a dual-head temporal convolutional network to a residual velocity and a 0.1 s axis-angle increment; the increments are integrated into a pose and compared with *insfilterMARG* and GPS-velocity integration.

Training uses Adam at 10^{-3}, minibatches of 256, eighty epochs, and a validation checkpoint. Early runs that checkpointed on ATE alone, or on a sum ATE + quat/30, kept the path and discarded the compass; the frozen weights, A8q, are the quaternion checkpoint of the same dual-head run, not a second architecture. Targets are standardized. The loss is mean squared error on the normalized residual velocity and on the axis-angle; a geodesic quaternion loss was left as a possible A-side ablation and was not used for the frozen row.

Integration at evaluation is the same for A and for the GPS-velocity baseline: the first TEST pose is taken from ground truth, the predicted one-second mean velocity is applied once per 1 s hop, i.e. every ten extracts, and the 0.1 s rotations are applied on every extract. ATE is the RMSE of the integrated NED position against ground truth on those 1 s samples; RTE is the mean magnitude of the 1 s relative displacement error; quaternion error is the

mean of 2 arccos(|q̂ · q|), in degrees, on the 0.1 s sequence [3], [4]. The EKF is scored on the same timestamps. GPS-velocity ATE on this TEST slice is 1.10 m, RTE 0.45 m; those two numbers are what "the network did something" has to beat, or at least tie, before anyone talks about beating the EKF. The full configuration used to reproduce A8q is collected in **Table 2**.

Table 2. Reproducibility settings for the defended A8q increment TCN: data, windowing, chronological split, architecture, and training.

Parameter	Value
Data and sensors	
Source trajectory	*LoggedQuadcopter*.mat (MATLAB)
Sensor models	*imuSensor*, gpsSensor
IMU sample rate	160 Hz
Input channels	12 (accelerometer 3, gyroscope 3, magnetometer 3, GPS velocity 3)
Windowing and targets	
Window length	1.0 s (160 samples)
Extraction hop	0.1 s (16 samples)
Integration / RTE step	1.0 s
Velocity target	residual on GPS velocity, δv
Orientation target	0.1 s axis-angle increment, δθ
Chronological split	
Train / validation / test	70 / 15 / 15 %
Guard gap between splits	10 windows (1.0 s)
Windows (train / val / test)	961 / 210 / 210 (21 at the 1 s step)
Architecture	
Input stem	Conv1d 12→64, kernel 5
Temporal blocks	4 residual TCN, 64 channels, kernel 5, dilations 1/2/4/8
Temporal pooling	concatenate(mean, last) → 128
Output heads	two MLPs, 128→64→3 (δv and δθ)
Training	
Normalization	per-channel z-score (train statistics), inputs and targets
Optimizer	Adam, learning rate 1×10^{-3}
LR schedule	ReduceLROnPlateau (patience 8, factor 0.5)
Batch size	256
Epochs	80
Loss	MSE(δv) + 1.0 · MSE(δθ)

Checkpoint	best validation quaternion (A8q)

4.4. Recursive and hybrid pilots

The B1 pilot reuses the A dataset and the same heads. Previous state is last velocity plus last quaternion, not NED position; scheduled sampling falls from 1 to 0 over the first 40% of epochs and stays at 0; training unrolls 40 hops, 4 s, and TEST is closed-loop from the first ground-truth pose. The C0 pilot concatenates the frozen EKF pose to the same twelve-channel window, predicts μ and log σ, and applies the mean residual independently at each window. Neither pilot retunes MATLAB Q or R. Their numbers belong in Chapter 6 and in the related-work discussion, not as replacements for A8q.

Three estimators can be built from the same MARG-plus-GPS-velocity window and the same LoggedQuadcopter log, and they are not three inventions, they are three answers to a single question: what is allowed to sit in the previous-state slot. The rejected answer is ground-truth pose at t−1. The remaining answers are: nothing (A), the network's own last prediction (B), and the EKF's pose (C).

5. RESULTS AND DISCUSSION

Summary: This chapter reports the increment-model ablations that produced the frozen A8q row, compares that row with insfilterMARG and with GPS-velocity integration on the same TEST slice, and then, as discussion rather than as a second defended method, records the closed-loop recursive pilot and the hybrid leftover-error pilot.

5.1. Results

The TEST slice is 121–141 s, twenty-one one-second samples after a one-second gap from validation, so the path that ATE sees is twenty seconds of flight, not a second holdout. All rows below use that slice, the same ground truth, and the same ATE / RTE / mean quaternion protocol. A8q is the frozen dual-head TCN, quaternion checkpoint; the A8 ATE checkpoint is shown only as the trade-off that was rejected, a better path with a dead compass. GPS-vel is not a second network, it is pos ← pos + v_gps·1.0 from the first TEST pose.

The same numbers are drawn in **Figure 3**; the path is **Figure 4**, the error traces are **Figure 5**, and the velocity that walks ATE is **Figure 6**.

Table 3. TEST metrics, 121–141 s, versus ground truth. A8q is the defended row. B1 and C0 are pilots.

Estimator	ATE (m)	RTE (m)	Quat (deg)
A8q (defended)	1.12	0.31	4.95
A8 ATE-ckpt (rejected)	0.93	0.32	14.3
GPS-vel integrate	1.10	0.45	—
insfilterMARG	0.78	0.20	7.46
B1 closed-loop (pilot)	1.42	0.30	2.09
C0 EKF+μ (pilot)	0.89	0.29	5.44

Figure 3 through **Figure 6** are the frozen A8q TEST plots; they use the same holdout as **Table 3**.

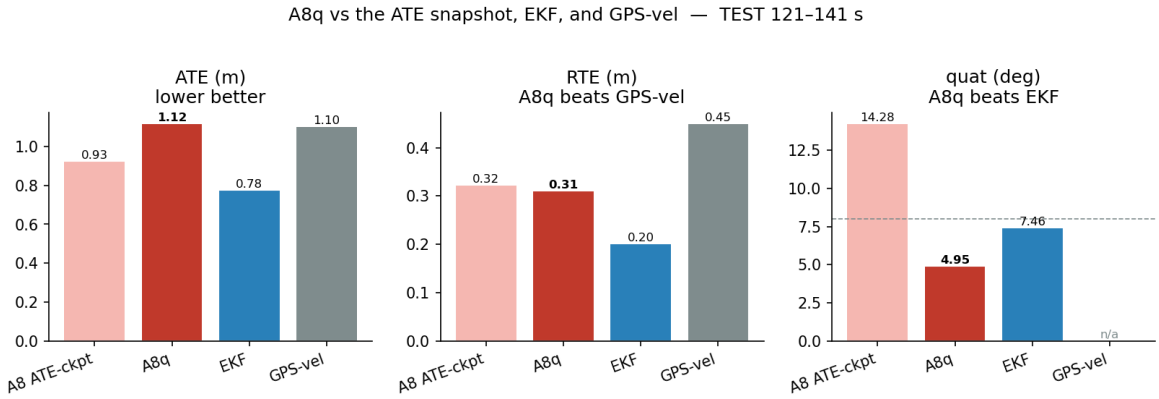


Figure 3. TEST scorecard. A8q (red) versus the rejected A8 ATE checkpoint, *insfilterMARG* and GPS-velocity integration. Quaternion is the number A8q wins; ATE is a tie with GPS-vel.

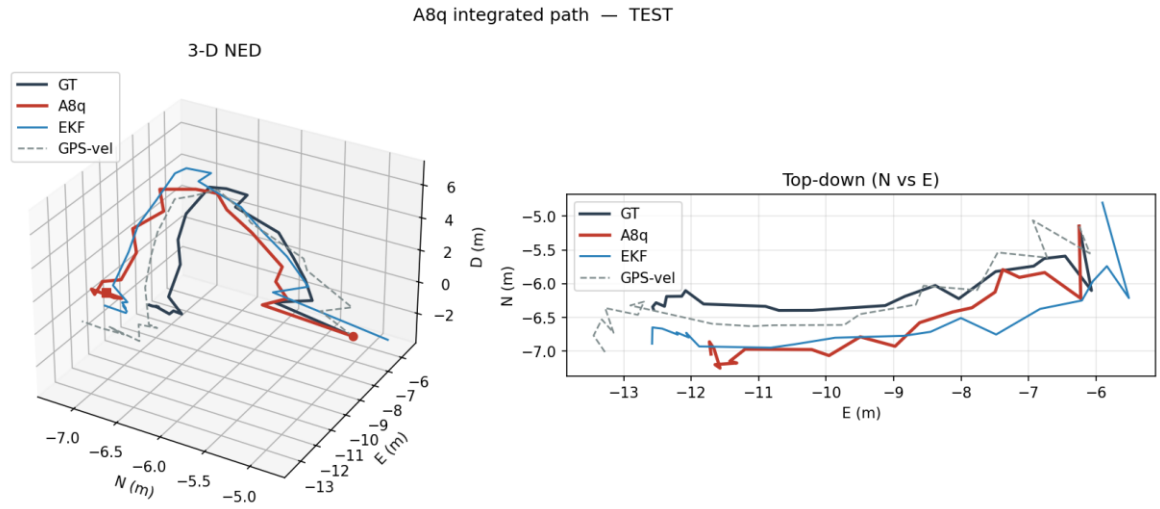


Figure 4. Integrated TEST path in NED and in a north-east top-down view. A8q follows the quadcopter; the EKF stays closer in position because it sees GPS LLA.

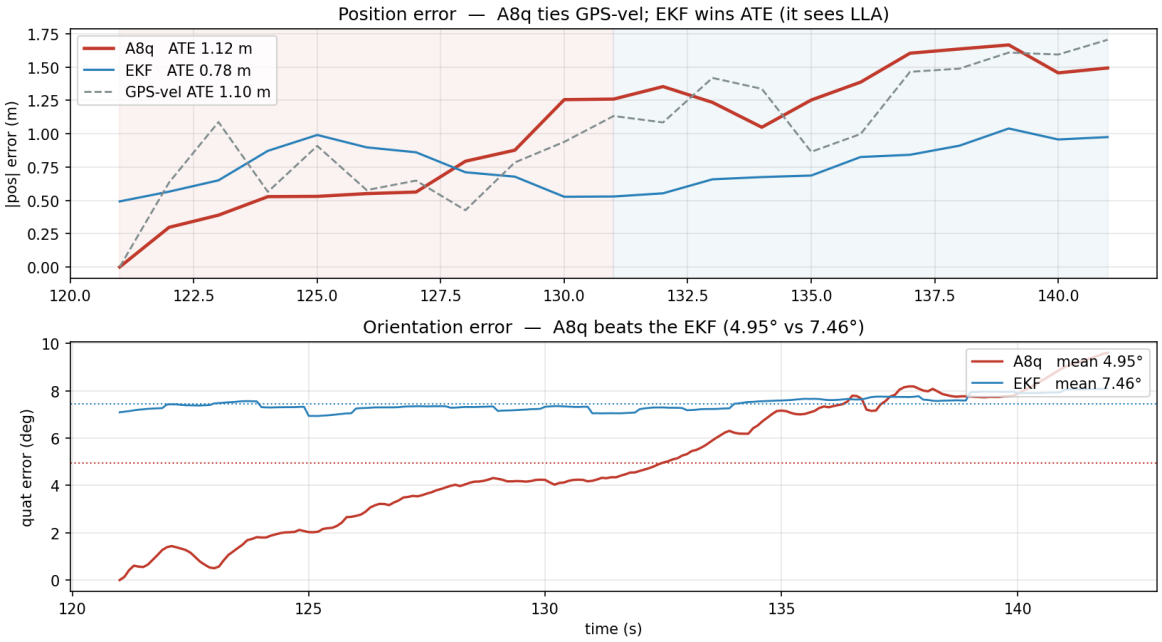


Figure 5. Position and quaternion error versus time. Position error grows from the first 10 s to the last 10 s; quaternion stays below the EKF for most of the holdout.

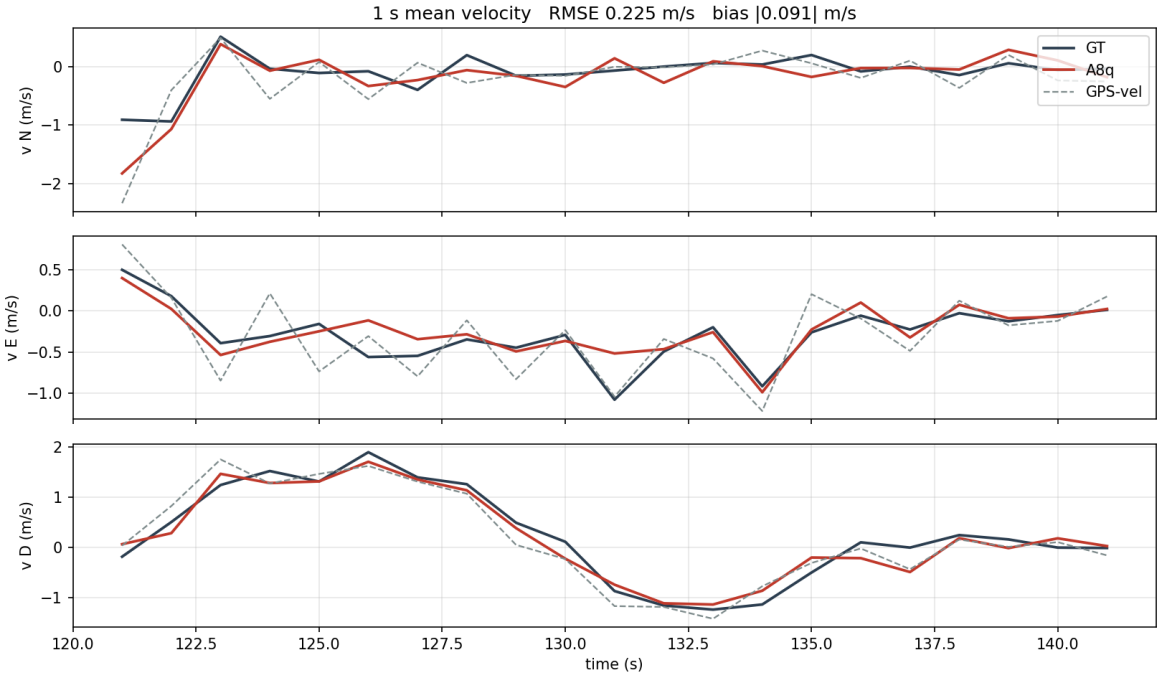


Figure 6. One-second mean velocity. The leftover bias of about 0.09 m/s is what walks ATE by a meter over the twenty-second TEST.

A0–A2 established that a 160 Hz unit quaternion increment is almost the identity and that a one-second axis-angle is too coarse; the usable orientation target is a 0.1 s rotation, and the usable position target is a one-second mean velocity, later a residual on GPS velocity.

A3–A7 showed the other failure mode: when velocity and rotation share one head, any ATE pressure spends the quaternion, and a one-second hop on a 142 s log starves the train set down to 97 windows. A8 split the heads and, for the first time, put ATE below GPS-vel on the ATE checkpoint, 0.93 m, at the cost of 14.3°; the quaternion checkpoint of that same run is A8q, 1.12 m ATE, 0.31 m RTE, 4.95°. A9 was a retrain of the same recipe and came back at 1.53 m ATE, 0.33 m RTE and 5.03°, which did not beat A8q, and that is why the row is frozen rather than re-rolled.

Splitting the holdout in half shows where the meter accumulates. A8q opens at 0.74 m ATE over the first ten seconds and closes at 1.41 m over the last ten, and the per-axis root-mean-square says the growth is not uniform: North and East start tight, 0.31 m and 0.32 m, and roughly double to 0.77 m and 0.76 m, while the Down channel is already the worst at 0.58 m and reaches 0.90 m, which is the expected signature of an estimator that never receives an altitude fix and therefore has nothing to pin the vertical. The EKF stays almost flat across the same split, 0.73 m to 0.80 m, because it is reading GPS position throughout; its per-axis error even trades a shrinking East term, 0.53 m down to 0.15 m, for a growing Down one, 0.26 m up to 0.60 m, so its total barely moves while A8q climbs on every axis.

The velocity head explains that climb without reference to the path at all. Its one-second mean velocity on TEST has a root-mean-square error of 0.22 m/s and a bias of only 0.09 m/s, i.e. the bias is well under half of the total, so most of what the head gets wrong is zero-mean hop-to-hop noise that RTE already credits it for keeping small at 0.31 m; what survives integration is the bias, and the bias is not isotropic, it is dominated by the North axis at -0.08 m/s against 0.03 and -0.04 on East and Down, so the drift has a direction rather than being a symmetric blur. It is the same meter seen from the velocity side: a fraction of a metre per second, held for twenty seconds, is a fraction of the path, and that is the honest ceiling of an open-loop velocity integrator that no amount of head tuning removes, only a position reference would.

5.2. Discussion

B1 is the first recursive run that is not a pose lookup. Closed-loop TEST is 1.42 m ATE, 0.30 m RTE and 2.09°, which is the best attitude in the project and a worse path than A8q; teacher-forced B1 is 1.26 m and 0.12°, an honest gap, unlike B0’s 0.07 m teacher score

that had been reading the NED position. That is the Chapter 6 sentence: a net that sees its last quaternion can beat A on heading and still lose the field.

C0 added a learned leftover to the frozen EKF and produced 0.89 m ATE, 0.29 m RTE and 5.44°. The attitude is better than the raw filter and worse than A8q; the path is worse than the filter it was sitting on, because the residual moved the EKF by about 0.7 m per sample and did so independently at each window. It can also look, carelessly, like a win over A8q on ATE; that comparison is the LLA leak again. The hand-tuned Q and R were already good on this log. This is why C stays a literature pilot.

The headline I am willing to defend is therefore narrow: on this simulated quadcopter, with this split, a dual-head increment TCN that never sees pose can beat *insfilterMARG* on orientation, tie GPS-velocity integration on ATE, and beat it on RTE, and the tempting alternatives, teacher-forced pose, recursive pose, and EKF residual, either lie, help heading at the expense of the path, or smear a filter that was already the position winner.

The difference between A8q and the EKF is not a mystery once the observations are written down. The filter sees GPS position; A8q does not; 0.78 m versus 1.12 m ATE is therefore not a fair fight, and I do not read it as the network losing a contest it was invited to. RTE is the fairer path metric, 0.31 m versus 0.20 m, close and still behind. Quaternion is the number A8q actually wins, 4.95° versus 7.46°, and it is also the number the ATE checkpoint threw away. Against GPS-vel the story is the opposite: ATE is a tie, 1.12 m versus 1.10 m, and RTE is a win, 0.31 m versus 0.45 m, i.e. the residual head did not invent a better mean speed than the satellite, it made the hops the right shape. A leftover velocity bias of about 0.09 m/s is still enough to walk a meter over the twenty-second holdout, which is why the last ten seconds are worse than the first, 1.41 m versus 0.74 m, and why I do not claim that ATE is solved.

One row in the table is finally a warning about the data rather than the model. A9 is A8's recipe trained a second time, the same twelve channels, the same chronological split, and it returned 1.53 m ATE where A8q sits at 1.12 m; a 0.41 m swing between two runs that differ only in initialization is itself larger than every ATE gap this table turns on, and it exists because the entire study lives on one 142 s trajectory whose training set is only 961 windows once the one-second hop is applied. I read A8q as the honest best of that recipe rather than a fortunate seed, which is why it is frozen and A9 is printed beside it instead of being quietly

dropped; but that variance is precisely why I keep the defended claim narrow, and why the real remedy, more flights and more varied motion, belongs to the next chapter rather than to this table.

6. CONCLUSIONS

Summary: This chapter restates what was asked of the increment estimator, what was actually shown on the simulated holdout, and which of the two unused forks, the recursive net and the hybrid leftover, are worth another student's time.

6.1. Conclusions

The general objective was met in the limited sense that a learned increment estimator exists, it does not consume ground-truth pose, it does not retrieve Q and R, and it has a TEST row against *insfilterMARG*. The quaternion half of the hypothesis holds on this log, 4.95° versus 7.46°; the ATE half holds only as a tie with GPS-velocity integration, not as a victory over the EKF, and I think that is the honest way to write it. The specific objectives on the dataset, the dual-head TCN, and the ATE/RTE protocol were carried through; the fourth objective, to document B and C without promoting them, is the reason those rows are not in the main claim. A trade-off remains, the more the checkpoint hunts ATE the faster the compass dies, or, conversely, the snapshot that keeps heading pays about 0.20 m of path versus the ATE specialist, and versus the EKF it pays more than that because the filter still owns GPS position. The work is simulated, single-trajectory, same-path chronological holdout; those are limitations, not footnotes.

The numbers that carry the claim are small and worth restating together. On the twenty-second TEST slice A8q integrates to 1.12 m ATE, 0.31 m RTE and 4.95° of mean quaternion error, against the frozen insfilterMARG at 0.78 m, 0.20 m and 7.46° and against open-loop GPS-velocity integration at 1.10 m and 0.45 m; read honestly, that is one clear win, orientation, where the network is about two and a half degrees better than the filter that is reading a magnetometer through a full MARG update, one genuine tie, ATE against the satellite velocity, and one win on the fairer path metric, RTE, where the residual head shaped the one-second hops better than the raw GPS velocity did. The position error is not uniform in time or in space: it opens at 0.74 m over the first ten seconds and reaches 1.41 m over the last ten, it is worst on the Down axis that no altitude fix ever pins, and it is driven by a velocity

bias of about 0.09 m/s that twenty seconds of integration turn into the better part of a metre, which is the ceiling I attribute to the open loop rather than to the architecture.

The strength of the result is mostly in what it does not do: A8q never sees its own pose, it never reads back the EKF's position, and it does not retune the filter's Q and R, so the row it posts is one that cannot be explained by state leakage, and that is the property the earlier course project could not claim. The weaknesses are equally concrete, and I would rather state them than let a committee find them: the whole study lives on a single simulated 142 s quadcopter trajectory, the holdout is the same path the model trained on rather than an independent flight, the training set is only 961 windows once the one-second hop is applied, and a plain retrain under the same recipe, A9, already drifted to 1.53 m, so the frozen row is the honest best of a recipe with real run-to-run variance rather than a settled number. None of these are fatal to the thesis as written, which is deliberately narrow, but all of them point at the same fix.

6.2. Future work

The recursive estimator is the line I would actually continue. B1 already shows that feeding last velocity and last quaternion, with scheduled sampling and a four-second unroll, can put closed-loop heading at 2.09° while giving back 0.30 m of ATE; the next useful B is a longer TEST, more than one flight, and, if anyone insists, a geodesic orientation loss, not another NED-position input. The hybrid leftover is not a priority, C0 over-corrected a filter whose Q and R were already adequate; a true *AI-IMU* rerun, new noise parameters written back into *insfilterMARG*, would only be interesting on a sensor that is not the example IMU. The hardware log from the soccer field is the other obvious extension, and it would force a ground-truth discussion that the geometric field model of the earlier project never settled. Several random seeds on A8q would also be more valuable than another architecture, A9 already showed that one retrain can miss the frozen row.

The thread through all of these is the one the results chapter kept returning to: the bottleneck is the data, not the backbone. A dual-head increment network is already enough to beat a classical filter on heading and to tie satellite velocity on the path without ever touching ground-truth pose; what it has not been asked to do is generalize across flights, and until it is given more than one trajectory and, eventually, the real soccer-field log, the right posture is

the one taken here, a narrow defended claim with its limitations written next to it rather than a headline that a single-path experiment cannot support.

BIBLIOGRAPHY

[1] [1] C. Chen, X. Lu, A. Markham, and N. Trigoni, "IONet: Learning to Cure the Curse of Drift in Inertial Odometry," in *Proc. AAAI Conf. Artif. Intell.*, 2018.

[2] [2] H. Yan, Q. Shan, and Y. Furukawa, "RIDI: Robust IMU Double Integration," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 621–636.

[3] [3] S. Herath, H. Yan, and Y. Furukawa, "RoNIN: Robust Neural Inertial Navigation in the Wild: Benchmark, Evaluations, and New Methods," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2020, pp. 3146–3152.

[4] [4] M. A. Esfahani, H. Wang, K. Wu, and S. Yuan, "OriNet: Robust 3-D Orientation Estimation With a Single Particular IMU," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 399–406, Apr. 2020.

[5] [5] W. Liu et al., "TLIO: Tight Learned Inertial Odometry," *IEEE Robot. Autom. Lett.*, vol. 5, no. 4, pp. 5653–5660, Oct. 2020.

[6] [6] M. Brossard, A. Barrau, and S. Bonnabel, "AI-IMU Dead-Reckoning," *IEEE Trans. Intell. Veh.*, vol. 5, no. 4, pp. 585–595, Dec. 2020.

[7] [7] M. K. Al-Sharman, Y. Zweiri, M. A. K. Jaradat, R. Al-Husari, D. Gan, and L. D. Seneviratne, "Deep-Learning-Based Neural Network Training for State Estimation Enhancement: Application to Attitude Estimation," *IEEE Trans. Instrum. Meas.*, vol. 69, no. 1, pp. 24–34, Jan. 2020.

[8] [8] J. Wang, S. Xu, Y. Xing, J. Lu, and H. Hu, "A Pseudo-Increment Prediction Method for GNSS/INS Integrated Navigation Based on Transformer–LSTM During GNSS Outages," in *Proc. IEEE VTC2025-Fall*, 2025.

[9] [9] C. Chen, B. Wang, C. X. Lu, N. Trigoni, and A. Markham, "A Survey on Deep Learning for Localization and Mapping: Towards the Age of Spatial Machine Intelligence," arXiv:2006.12567, 2020.

[10] [10] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *J. Basic Eng.*, vol. 82, no. 1, pp. 35–45, 1960.

[11] [11] P. D. Groves, *Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems*, 2nd ed. Norwood, MA, USA: Artech House, 2013.

[12] [12] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

[13] [13] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[14] [14] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proc. ICLR*, 2015.

[15] [15] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proc. IEEE,* vol. 86, no. 11, pp. 2278–2322, 1998.

[16] [16] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *J. Mach. Learn. Res.*, vol. 15, pp. 1929–1958, 2014.

[17] [17] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, "Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2015.

[18] [18] The MathWorks, Inc., "insfilterMARG: Estimate pose using MARG and GPS data," *Sensor Fusion and Tracking Toolbox* documentation, Natick, MA, USA.

APPENDIX A. Simulated MARG/GPS dataset

The learning table is produced by option_a/make_dataset.m. The script loads LoggedQuadcopter.mat, runs imuSensor and gpsSensor at 160 Hz and 1 Hz, and writes dataset.csv (accelerometer, gyroscope, magnetometer, GPS velocity, ground-truth NED position and quaternion) together with filterstate.csv (insfilterMARG pose). GPS latitude, longitude and altitude are used only inside the EKF and are not copied into the twelve-channel stream. A one-second warmup is dropped. The MATLAB Sensor Fusion and Tracking Toolbox is required at runtime; helper scopes live in matlab_helpers/.

```
% Option A dataset: noisy MARG + GPS vel, GT pose, and insfilterMARG log.
% Rows: accel(3) gyro(3) mag(3) gpsvel(3) gt_pos(3) gt_quat_wxyz(4)

clear; clc; close all;

thisDir = fileparts(mfilename('fullpath'));

imuFs = 160;
gpsFs = 1;
refloc = [42.2825 -72.3430 53.0352];
secondsToSim = 142;
warmup = 160;
seed = 1;

imuSamplesPerGPS = imuFs / gpsFs;
assert(imuSamplesPerGPS == fix(imuSamplesPerGPS));

trajFile = fullfile(thisDir, 'LoggedQuadcopter.mat');
if ~isfile(trajFile)
    error('Missing %s', trajFile);
end
load(trajFile, 'trajData');
trajOrient = trajData.Orientation;
trajVel    = trajData.Velocity;
trajPos    = trajData.Position;
trajAcc    = trajData.Acceleration;
trajAngVel = trajData.AngularVelocity;

rng(seed);

gps = gpsSensor('UpdateRate', gpsFs);
gps.ReferenceLocation = refloc;
gps.DecayFactor = 0.5;
gps.HorizontalPositionAccuracy = 1.6;
gps.VerticalPositionAccuracy = 1.6;
gps.VelocityAccuracy = 0.1;

imu = imuSensor('accel-gyro-mag', 'SampleRate', imuFs);
imu.MagneticField = [19.5281 -5.0741 48.0067];
imu.Accelerometer.MeasurementRange = 19.6133;
imu.Accelerometer.Resolution = 0.0023928;
imu.Accelerometer.ConstantBias = 0.19;
imu.Accelerometer.NoiseDensity = 0.0012356;
imu.Gyroscope.MeasurementRange = deg2rad(250);
imu.Gyroscope.Resolution = deg2rad(0.0625);
imu.Gyroscope.ConstantBias = deg2rad(3.125);
imu.Gyroscope.AxesMisalignment = 1.5;
```

```
imu.Gyroscope.NoiseDensity = deg2rad(0.025);
imu.Magnetometer.MeasurementRange = 1000;
imu.Magnetometer.Resolution = 0.1;
imu.Magnetometer.ConstantBias = 100;
imu.Magnetometer.NoiseDensity = 0.3 / sqrt(50);


fusionfilt = insfilterMARG;
fusionfilt.IMUSampleRate = imuFs;
fusionfilt.ReferenceLocation = refloc;


initstate = zeros(22, 1);
initstate(1:4)   = compact(meanrot(trajOrient(1:100)));
initstate(5:7)   = mean(trajPos(1:100, :), 1);
initstate(8:10)  = mean(trajVel(1:100, :), 1);
initstate(11:13) = imu.Gyroscope.ConstantBias ./ imuFs;
initstate(14:16) = imu.Accelerometer.ConstantBias ./ imuFs;
initstate(17:19) = imu.MagneticField;
initstate(20:22) = imu.Magnetometer.ConstantBias;
fusionfilt.State = initstate;


Rmag = 0.0862;
Rvel = 0.0051;
Rpos = 5.169;
fusionfilt.AccelerometerBiasNoise = 0.010716;
fusionfilt.AccelerometerNoise = 9.7785;
fusionfilt.GyroscopeBiasNoise = 1.3436e-14;
fusionfilt.GyroscopeNoise = 0.00016528;
fusionfilt.MagnetometerBiasNoise = 2.189e-11;
fusionfilt.GeomagneticVectorNoise = 7.67e-13;
fusionfilt.StateCovariance = 1e-9 * eye(22);


loopBound = floor(secondsToSim * imuFs / imuFs) * imuFs;
pqorient = quaternion.zeros(loopBound, 1);
pqpos = zeros(loopBound, 3);
dataset = zeros(loopBound, 19);


gpsvel = zeros(1, 3);
mag_hold = zeros(1, 3);
fcnt = 1;


while fcnt <= loopBound
    for ff = 1:imuSamplesPerGPS
        [accel, gyro, mag] = imu(trajAcc(fcnt, :), trajAngVel(fcnt, :), ...
                                 trajOrient(fcnt));
        [qw, qx, qy, qz] = parts(trajOrient(fcnt, :));
        dataset(fcnt, :) = [accel, gyro, mag_hold, gpsvel, ...
                            trajPos(fcnt, :), qw, qx, qy, qz];


        predict(fusionfilt, accel, gyro);
        [fusedPos, fusedOrient] = pose(fusionfilt);
        pqorient(fcnt) = fusedOrient;
        pqpos(fcnt, :) = fusedPos;
        fcnt = fcnt + 1;
    end
    if fcnt <= size(trajPos, 1)
        [lla, gpsvel] = gps(trajPos(fcnt, :), trajVel(fcnt, :));
        fusegps(fusionfilt, lla, Rpos, gpsvel, Rvel);
        fusemag(fusionfilt, mag, Rmag);
        mag_hold = mag;
    end
end


keep = (warmup + 1):loopBound;
t = ((keep - 1) / imuFs).';
fq = compact(pqorient(keep));
```

```
gq = compact(trajOrient(keep));

writematrix(dataset(keep, :), fullfile(thisDir, 'dataset.csv'));
writematrix([t, pqpos(keep, :), fq, trajPos(keep, :), gq], ...
    fullfile(thisDir, 'filterstate.csv'));

posd = pqpos(keep, :) - trajPos(keep, :);
quatd = rad2deg(dist(pqorient(keep), trajOrient(keep)));
msep = sqrt(mean(posd.^2));
fprintf('Wrote %d rows to dataset.csv and filterstate.csv\n', numel(keep));
fprintf('EKF pos RMSE  X %.2f  Y %.2f  Z %.2f m\n', msep(1), msep(2), msep(3));
fprintf('EKF quat RMSE %.2f deg\n', sqrt(mean(quatd.^2)));
```

APPENDIX B. Training and plotting notebooks

The defended increment model is trained in option_a/train.ipynb; the frozen weights are option_a_a8q.pth. Figures 1–4 in Chapter 5 are produced by option_a/plot_a8q.ipynb from results_a8q.npz and results_a8q.json, without retraining. The recursive and hybrid pilots live in option_b/train.ipynb and option_c/train.ipynb and are not the defended method.

```
# option_a/train.ipynb — defended increment TCN (A8q)
# Constants, quaternion helpers, dual-head model, and closed-loop integrate.

IMU_FS = 160
WIN_SEC, EXTRACT_SEC, HOP_SEC, RTE_SEC = 1.0, 0.1, 1.0, 1.0
W = int(WIN_SEC * IMU_FS)
STRIDE = int(EXTRACT_SEC * IMU_FS)
HOP_N = int(round(HOP_SEC / EXTRACT_SEC))
BATCH_SIZE, EPOCHS, LR = 256, 80, 1e-3
LAMBDA_ROT = 1.0
VAL_FRAC, TEST_FRAC, GAP = 0.15, 0.15, HOP_N

def q_normalize(q, eps=1e-8):
    return q / (np.linalg.norm(q, axis=-1, keepdims=True) + eps)

def q_conj(q):
    o = q.copy(); o[..., 1:] *= -1; return o

def q_mul(a, b):
    aw, ax, ay, az = (a[..., k] for k in range(4))
    bw, bx, by, bz = (b[..., k] for k in range(4))
    return np.stack([
        aw*bw - ax*bx - ay*by - az*bz,
        aw*bx + ax*bw + ay*bz - az*by,
        aw*by - ax*bz + ay*bw + az*bx,
        aw*bz + ax*by - ay*bx + az*bw,
    ], axis=-1)

def quat_to_rotvec(q):
    q = q_normalize(q)
    q = np.where(q[..., :1] < 0, -q, q)
    w = np.clip(q[..., 0], -1, 1)
    xyz = q[..., 1:]
    n = np.linalg.norm(xyz, axis=-1, keepdims=True)
    angle = 2 * np.arccos(w)
    axis = xyz / np.clip(n, 1e-8, None)
    return (axis * angle[..., None]).astype(np.float32)

def rotvec_to_quat(v, eps=1e-8):
    angle = np.linalg.norm(v, axis=-1, keepdims=True)
    axis = v / np.clip(angle, eps, None)
    half = 0.5 * angle
    return np.concatenate([np.cos(half), axis * np.sin(half)], -1).astype(np.float32)

class TcnBlock(nn.Module):
    def __init__(self, ch, k, d):
        super().__init__()
        pad = (k - 1) * d // 2
        self.f = nn.Sequential(
            nn.Conv1d(ch, ch, k, padding=pad, dilation=d),
            nn.ReLU(),
            nn.Conv1d(ch, ch, k, padding=pad, dilation=d),
        )
        self.act = nn.ReLU()
```

```
    def forward(self, x):
        return self.act(x + self.f(x))

class IncModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.stem = nn.Conv1d(12, 64, 5, padding=2)
        self.tcn = nn.Sequential(
            TcnBlock(64, 5, 1),
            TcnBlock(64, 5, 2),
            TcnBlock(64, 5, 4),
            TcnBlock(64, 5, 8),
        )
        self.head_v = nn.Sequential(nn.Linear(128, 64), nn.ReLU(), nn.Linear(64, 3))
        self.head_q = nn.Sequential(nn.Linear(128, 64), nn.ReLU(), nn.Linear(64, 3))
    def forward(self, x):
        z = torch.relu(self.stem(x))
        z = self.tcn(z)
        h = torch.cat([z.mean(-1), z[:, :, -1]], 1)
        return torch.cat([self.head_v(h), self.head_q(h)], 1)

def ate_rte(est, ref, hop=HOP_SEC, sec=RTE_SEC):
    ate = float(np.sqrt(((est - ref) ** 2).sum(1).mean()))
    w = max(int(round(sec / hop)), 1)
    rel = [
        np.linalg.norm((est[i + w] - est[i]) - (ref[i + w] - ref[i]))
        for i in range(0, len(est) - w, max(w // 2, 1))
    ]
    return ate, float(np.mean(rel)) if rel else float("nan")

def integrate_pos(vel, pos0):
    n = len(vel)
    pos_hat = np.zeros((n, 3), np.float32)
    pos_hat[0] = pos0
    for t in range(1, n):
        pos_hat[t] = pos_hat[t - 1] + vel[t] * HOP_SEC
    return pos_hat

def integrate_q(drot, q0):
    n = len(drot)
    q_hat = np.zeros((n, 4), np.float32)
    q_hat[0] = q0
    for t in range(1, n):
        q_hat[t] = q_normalize(q_mul(q_hat[t - 1], rotvec_to_quat(drot[t])))
    return q_hat
```